RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

RAJALAKSHMI ENGINEERING COLLEGE

AD23B31 - Image Processing and Computer Vision

LABORATORY RECORD NOTEBOOK

NAME: RENITH JOEL R R

YEAR/SEMESTER: 3RD YEAR/6TH

BRANCH: COMPUTER SCIENCE AND ENGINEERING

REGISTER NO: 2116230701263

COLLEGE ROLL NO: 230701263

ACADEMIC YEAR: 2025 - 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

BONAFIDE CERTIFICATE

NAME: RENITH JOEL R R **BRANCH/SECTION:CSE-D**

ACADEMIC YEAR:2025-2026 **SEMESTER: 6**

REGISTER NO: **2116230701263**

Certified that this is a Bonafide record of work done by the above student in the **AD23B31 - Image Processing and Computer Vision** during the year 20 - 20

Signature of Faculty In-charge

Submitted for the Practical Examination Held on:

Internal Examiner **External Examiner**

INDEX

EX.NO	DATE	NAME OF THE EXPERIMENT	PAGE NO	STAFF SIGN
1	10/1/2026	Installation of Open CV and to perform the basic image handling processing operation on the image		
2	24/1/2026	2D Image Transformation		
3	31/1/2026	Implement Edge Detection, Line Detection and Corner Detection		
4	7/2/2026	Demonstrate Camera Calibration using python		
5	14/2/2026	Implement Image Histogram and Histogram Equalization		
6	21/2/2026	Develop a python program for Skin Color Detection		
7	7/3/2026	Create a python program for Warping and Estimation		
8	14/3/2026	Develop a python program for Motion Tracking		
9	4/4/2026	Design a program for Object Detection using YOLO		
10	11/4/2026	Develop a python program for Stereo Vision and Depth Estimation		
11	25/4/2026	Demonstrate Augmented Reality using Feature Matching		
12	25/4/2026	Mini Project		

EXPERIMENT 1:

```
import cv2 import
matplotlib.pyplot as plt

cb_img=cv2.imread("Downloads/download.jpg",cv2.IMREAD_GRAYSCALE)
print(cb_img) plt.imshow(cb_img,cmap='gray')

plt.axis('off') plt.show()
```

```
[[ 67  67  68 ...  59  59  59]
 [ 67  68  68 ...  59  59  59]
 [ 68  68  68 ...  59  59  59]
 ...
 [141 144 147 ...  98  98  98]
 [141 144 147 ...  94  94  94]
 [141 143 146 ...  93  93  93]]
```

```
print(cb_img) print("Shape of the
image:",cb_img.shape) print("Data type of the
image:",cb_img.dtype) print("Size of the
image:",cb_img.size)
plt.imshow(cb_img,cmap='gray')

plt.axis('off') plt.show()
```

```
[[ 67  67  68 ...  59  59  59]
 [ 67  68  68 ...  59  59  59]
 [ 68  68  68 ...  59  59  59]
 ...
 [141 144 147 ...  98  98  98]
 [141 144 147 ...  94  94  94]
 [141 143 146 ...  93  93  93]]
Shape of the image: (148, 204)
Data type of the image: uint8
Size of the image: 30192
```

img=cv2.imread("Downloads/download.jpg") print("Image shape (height,width,channels) is:",img.shape) print("Image size is:",img.size) print("Data type of image is:",img.dtype)

```
Image shape (height,width,channels) is: (148, 204, 3)
Image size is: 90576
Data type of image is: uint8
```

flower=cv2.imread("Downloads/download.jpg",cv2.IMREAD_COLOR) plt.imshow(flower)

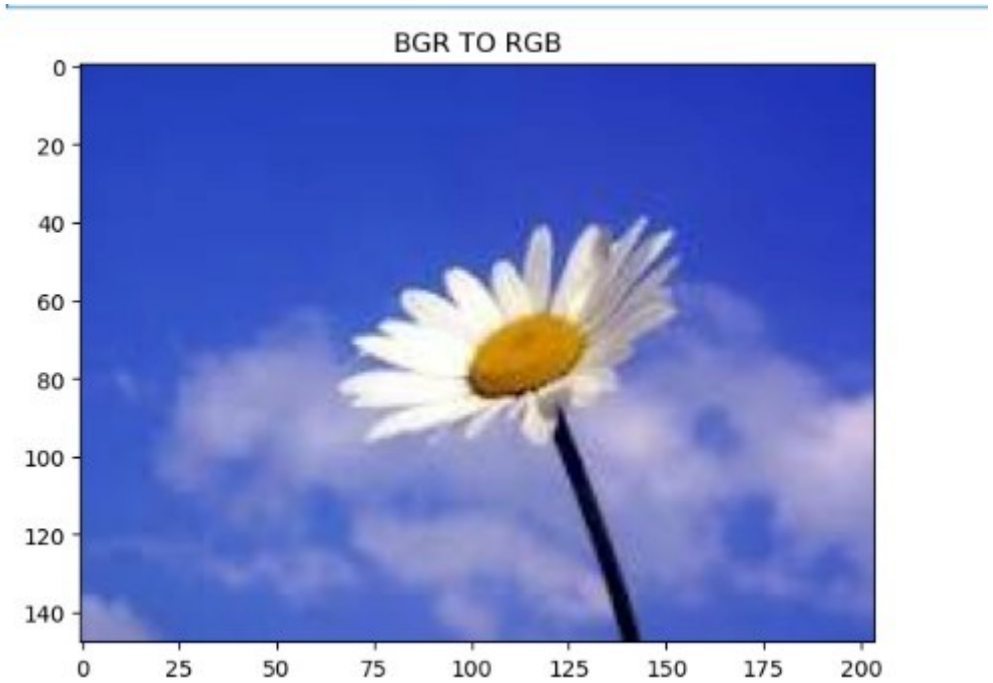
plt.axis('off') plt.show()

```
reverse=flower[:,:,::-1]
plt.imshow(reverse)
plt.axis('off') plt.show()
```

```
b,g,r=cv2.split(flower)
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(r,cmap="gray")
plt.title("Red channel")
plt.axis('off') plt.subplot(1,3,2)
plt.imshow(g,cmap='gray')
plt.title("Green Channel")
plt.axis('off') plt.subplot(1,3,3)
plt.imshow(b,cmap='gray')
```

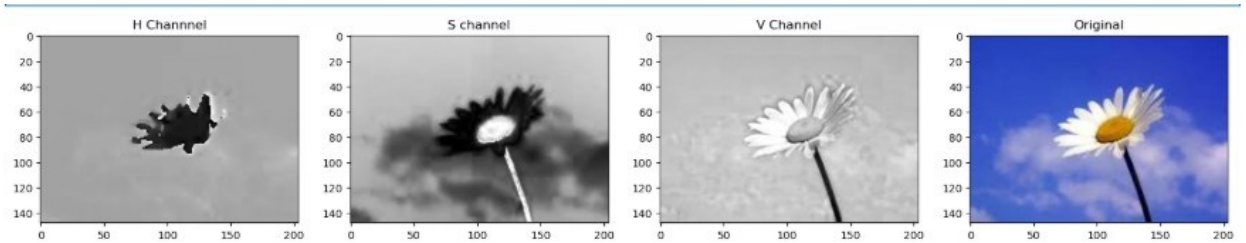
plt.title("Blue Channel")
plt.axis('off') plt.show()

flower=cv2.imread("Downloads/download.jpg",cv2.IMREAD_COLOR)
flower1=cv2.cvtColor(flower,cv2.COLOR_BGR2RGB) plt.imshow(flower1) plt.title("BGR TO RGB") plt.show()

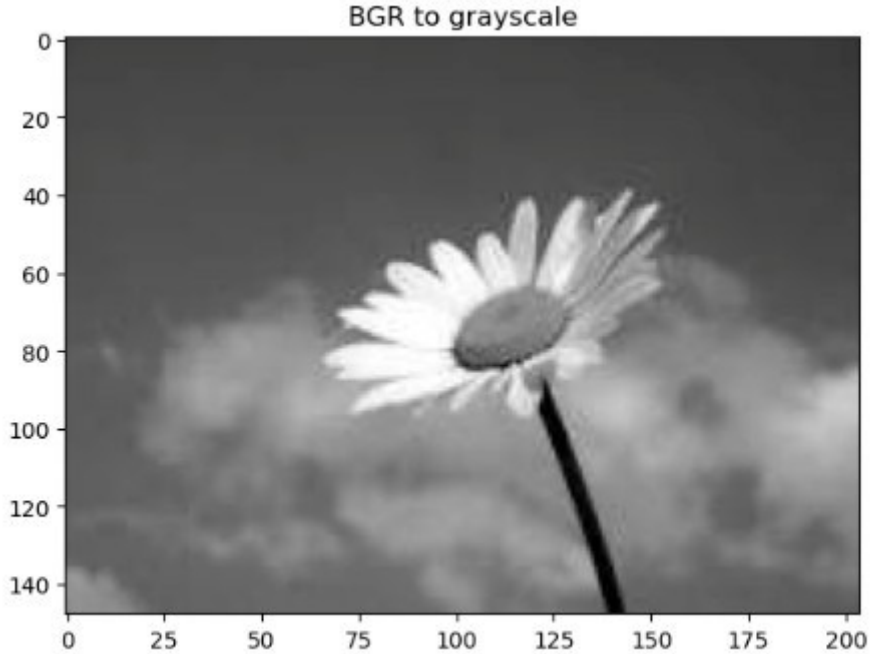


hsv=cv2.cvtColor(flower,cv2.COLOR_BGR2HSV) h,s,v=cv2.split(hsv)
plt.figure(figsize=[20,5]) plt.subplot(1,4,1) plt.imshow(h,cmap="gray") plt.title("H Channnel")

plt.subplot(1,4,2) plt.imshow(s,cmap="gray") plt.title("S channel")
plt.subplot(1,4,3) plt.imshow(v,cmap="gray") plt.title("V Channel") plt.subplot(1,4,4)
plt.imshow(cv2.cvtColor(flower,cv2.COLOR_BGR2RGB))
plt.title("Original") plt.show()

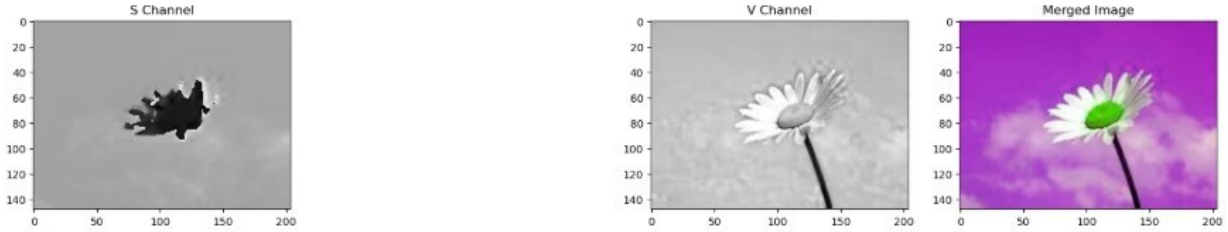


flower=cv2.imread("Downloads/download.jpg",cv2.IMREAD_COLOR)
flower1=cv2.cvtColor(flower,cv2.COLOR_BGR2GRAY) plt.imshow(flower1,cmap="gray") plt.title("BGR to grayscale") plt.show()



flower=cv2.imread("Downloads/download.jpg",cv2.IMREAD_COLOR)
hsv=cv2.cvtColor(flower,cv2.COLOR_BGR2HSV)

h,s,v=cv2.split(hsv) h_new=h+30 flower_merged=cv2.merge((h_new,s,v))
flower_rgb=cv2.cvtColor(flower_merged,cv2.COLOR_HSV2RGB)
h,s,v=cv2.split(flower_merged) plt.figure(figsize=[20,5]) plt.subplot(1,4,1)
plt.imshow(h,cmap="gray") plt.title("S Channel") plt.subplot(1,4,3)
plt.imshow(v,cmap='gray') plt.title("V Channel") plt.subplot(1,4,4)
plt.imshow(flower_rgb) plt.title("Merged Image") plt.show()



flower=cv2.imread("Downloads/download.jpg",cv2.COLOR_BGR2RGB) cv2.imwrite("saveflower.jpg",flower1)

```
True
```

EXPERIMENT 2:

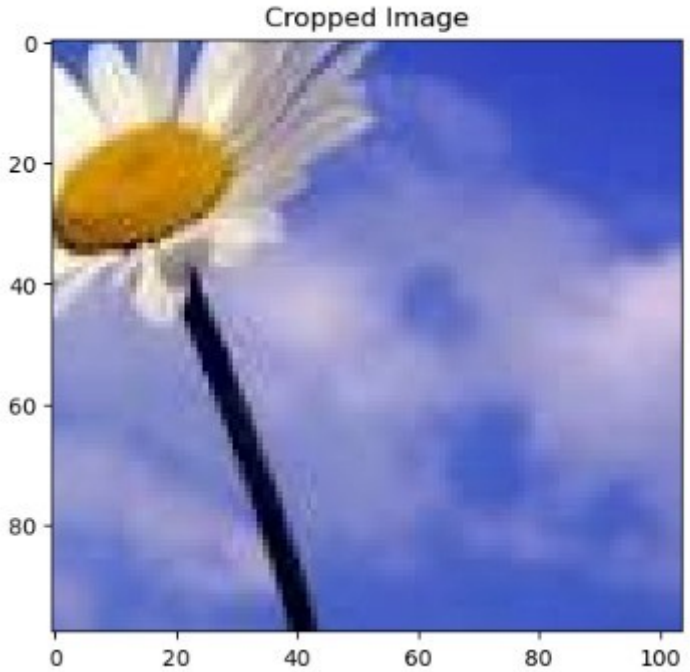
```
import numpy as np import matplotlib.pyplot as plt
img=cv2.imread("Downloads/download.jpg")
img_rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb) plt.title("Original Image") plt.axis("off")
```

```
(np.float64(-0.5), np.float64(203.5), np.float64(147.5), np.float64(-0.5))
```

Original Image

```
cropped=img_rgb[50:300 , 100:400]

#cropped_rgb=cv2.cvtColor(cropped,cv2.COLOR_BGR2RGB)
plt.imshow(cropped) plt.title("Cropped Image")

#plt.axis("off")
```

```
Text(0.5, 1.0, 'Cropped Image')
```



resized=cv2.resize(img_rgb,(300,300))
plt.imshow(resized) plt.title("Resized Image")
plt.axis("off")

```
(np.float64(-0.5), np.float64(299.5), np.float64(299.5), np.float64(-0.5))
```

rows,cols=img.shape[:2] M=np.float32([[1,0,50],[0,1,30]])
translated=cv2.warpAffine(img,M,(cols,rows))

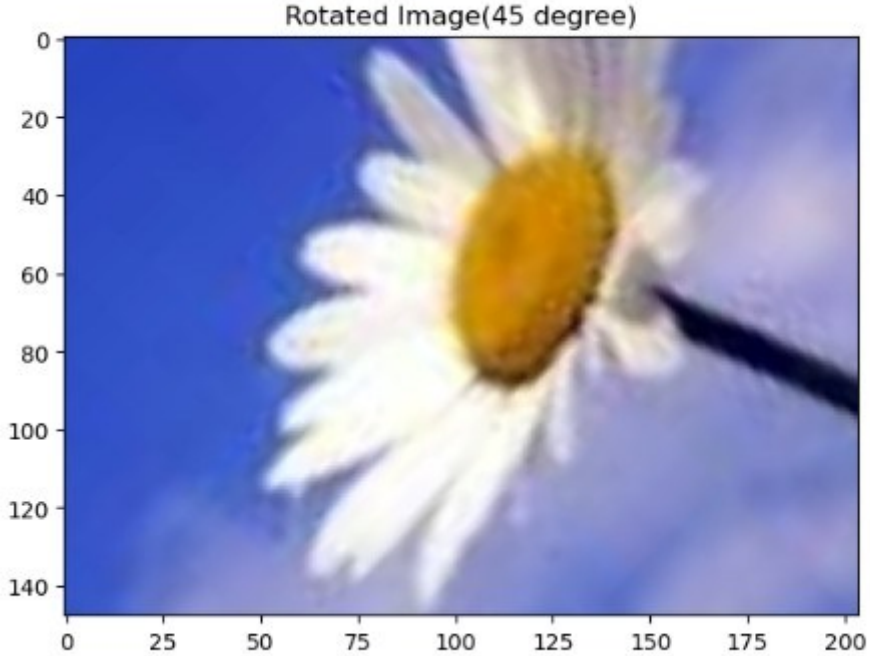
translated_rgb=cv2.cvtColor(translated,cv2.COLOR_BGR2RGB)
plt.imshow(translated_rgb) plt.title("Translated Image")

#plt.axis("off")

```
Text(0.5, 1.0, 'Translated Image')
```

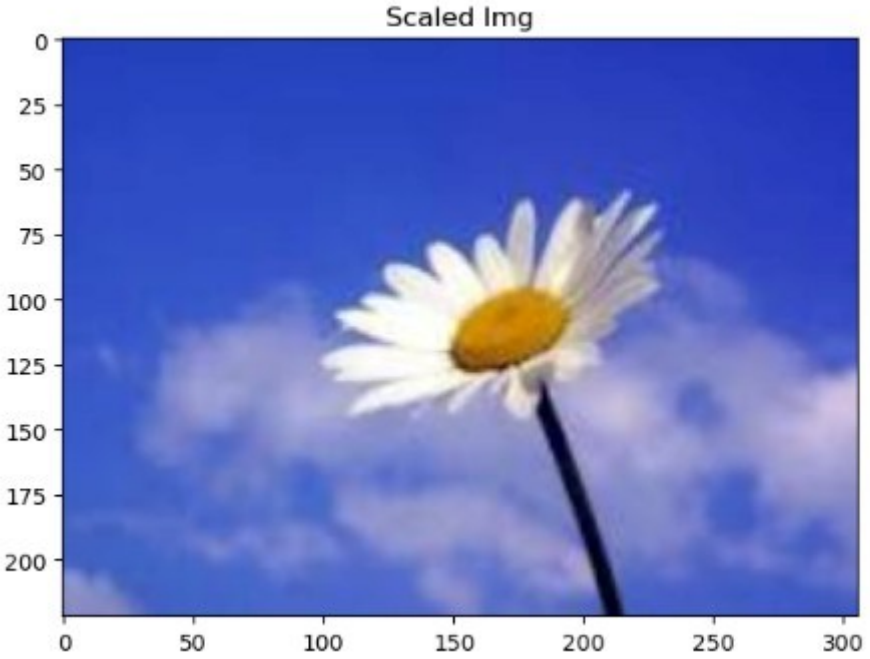
center=(cols//2,rows//2) #floor division M=cv2.getRotationMatrix2D(center,45,2.0)
rotated=cv2.warpAffine(img,M,(cols,rows))
rotated_rgb=cv2.cvtColor(rotated,cv2.COLOR_BGR2RGB) plt.imshow(rotated_rgb)
plt.title("Rotated Image(45 degree)")

```
Text(0.5, 1.0, 'Rotated Image(45 degree)')
```




```
scaled = cv2.resize(img,None,fx=1.5,fy=1.5)
scaled_rgb=cv2.cvtColor(scaled,cv2.COLOR_BGR2RGB)
plt.imshow(scaled_rgb) plt.title("Scaled Img")
```

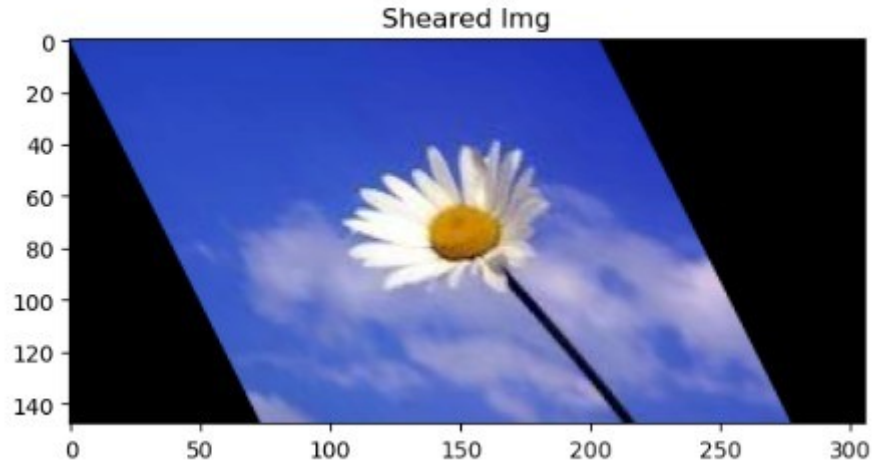
```
Text(0.5, 1.0, 'Scaled Img')
```




```
rows,cols=img.shape[:2]

M=np.float32([[1,0.5,0],[0,1,0]])
sheared=cv2.warpAffine(img,M,(int(cols*1.5),rows))
```

sheared_rgb=cv2.cvtColor(sheared,cv2.COLOR_BGR2RGB)
plt.imshow(sheared_rgb) plt.title("Sheared Img")

```
Text(0.5, 1.0, 'Sheared Img')
```



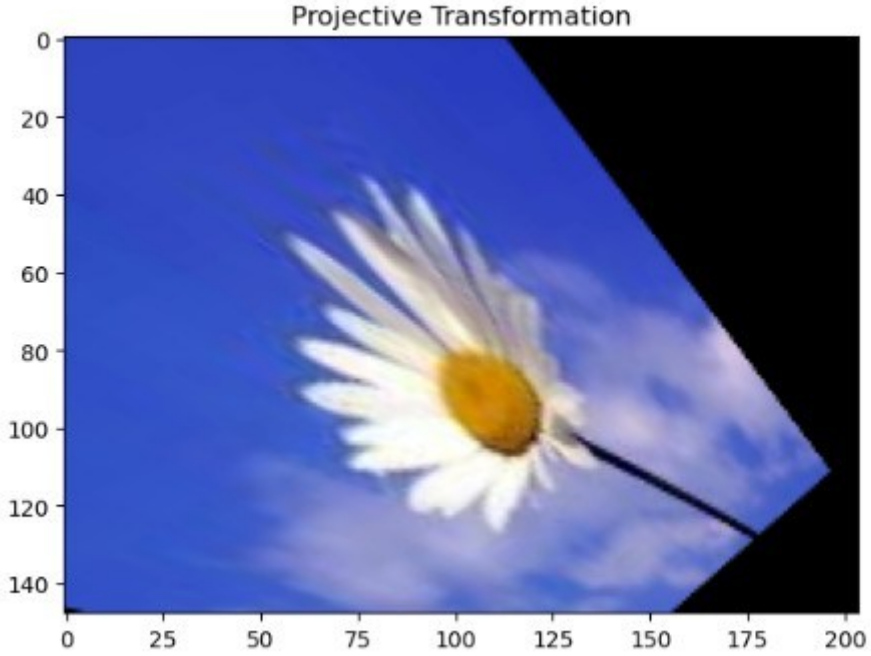
flipped=cv2.flip(img,1)
flipped_rgb=cv2.cvtColor(flipped,cv2.COLOR_BGR2RGB)
plt.imshow(flipped_rgb) plt.title("Reflected Img(horizontal flip)")

```
Text(0.5, 1.0, 'Reflected Img(horizontal flip)')
```

rows,cols=img.shape[:2] pts1=np.float32([[50,50],[cols-50,50],[50,rows-50],[cols-50,rows-50]])
pts2=np.float32([[10,100],[cols-100,50],[100,rows-10],[cols-50,rows-50]]) M=cv2.getPerspectiveTransform(pts1,pts2)
perspective=cv2.warpPerspective(img,M,(cols,rows))

```
perspective_rgb=cv2.cvtColor(perspective,cv2.COLOR_BGR2RGB) plt.imshow(perspective_rgb) plt.title("Projective Transformation")
```

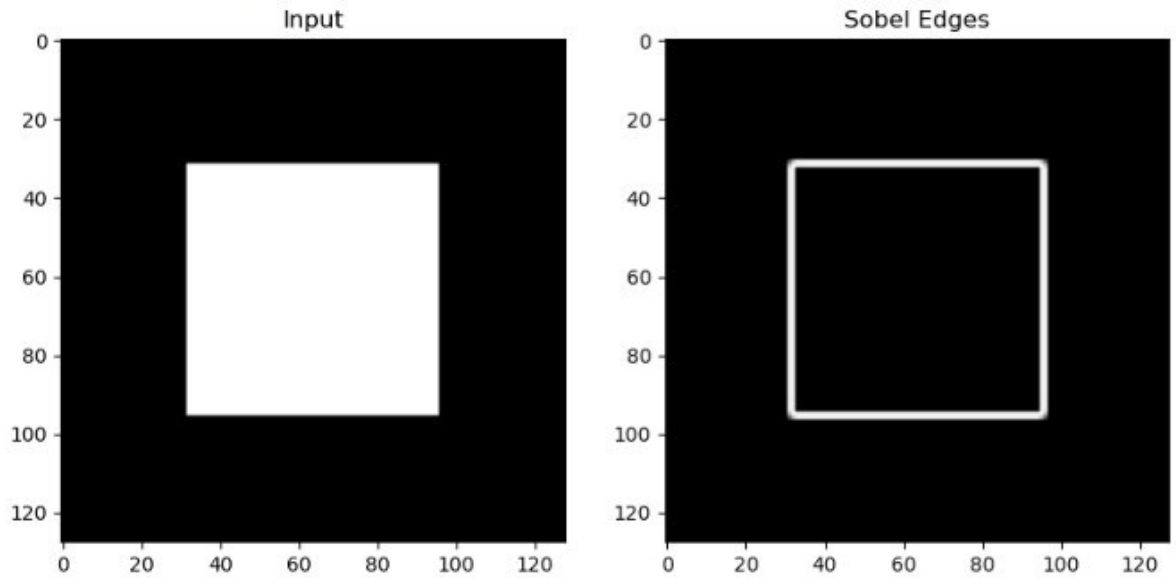
```
Text(0.5, 1.0, 'Projective Transformation')
```



EXPERIMENT 3:

```
import numpy as np from scipy import
ndimage import matplotlib.pyplot as plt
import cv2 def
sobel_edge_detection(image):

   sx=np.array([[-1,0,1],[-2,0,2],[-1,0,1]])     sy=np.array([[-1,-2,-
1],[0,0,0],[1,2,1]])

   Ix=ndimage.convolve(image,sx,mode='constant')
Iy=ndimage.convolve(image,sy,mode='constant')     magnitude=np.hypot(Ix,Iy)
return magnitude/magnitude.max() img=np.zeros((128,128))
img[32:96,32:96]=1.0 edges=sobel_edge_detection(img)
plt.figure(figsize=(10,5))
plt.subplot(121);plt.imshow(img,cmap='gray');plt.title("Input")
plt.subplot(122);plt.imshow(edges,cmap='gray');plt.title("Sobel Edges") plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```

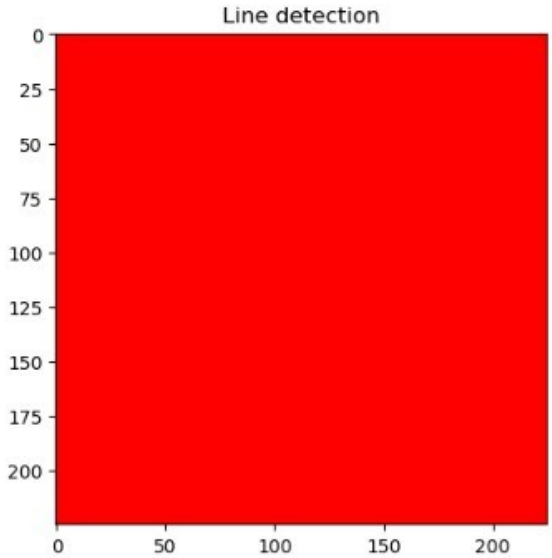



```
def
hough_line_detection(binary_edges,theta_step=0.5,threshold_ratio=0.6):
   h,w=binary_edges.shape     diag=int(np.ceil(np.hypot(h,w)))
rhos=np.linspace(-diag,diag,2*diag+1)
thetas=np.deg2rad(np.arange(-90,90,theta_step))
accumulator=np.zeros((len(rhos),len(thetas)),dtype=int)
y,x=np.nonzero(binary_edges>0.1)

   for px,py in zip(x,y):        for t_idx , theta in enumerate(thetas):
rho=px*np.cos(theta)+py*np.sin(theta)
rho_idx=np.argmin(np.abs(rhos-rho))
accumulator[rho_idx,t_idx]+=1     max_votes=accumulator.max()
```

```
peaks=np.argwhere(accumulator>=threshold_ratio*max_votes)     lines=[]
for rho_idx,theta_idx in peaks:

      lines.append((rhos[rho_idx],thetas[theta_idx]))     return
lines,accumulator


img=cv2.imread("Downloads/lion_line.jpg")
edges=cv2.Canny(img,100,200)

lines,accumulator=hough_line_detection(edges)
print("Detected Lines (rho,theta):",lines)
```

```
Detected Lines (rho,theta): [(np.float64(-105.0), np.float64(-1.53588974175501)), (np.float64(-88.0), np.float64(-1.5707963267948966)), (np.float64(-8
7.0), np.float64(-1.4486232791552935)), (np.float64(-86.0), np.float64(-1.5533430342749532)), (np.float64(-85.0), np.float64(-1.5707963267948966)), (n
p.float64(-84.0), np.float64(-1.53588974175501)), (np.float64(-81.0), np.float64(-1.5533430342749532)), (np.float64(-81.0), np.float64(-1.5358897417550
1)), (np.float64(-80.0), np.float64(-1.5446163880149817)), (np.float64(-80.0), np.float64(-1.5271630954950384)), (np.float64(-78.0), np.float64(-1.5358
8974175501)), (np.float64(-77.0), np.float64(-1.5271630954950384)), (np.float64(-77.0), np.float64(-1.4049900478554354)), (np.float64(-76.0), np.float6
4(-1.5707963267948966)), (np.float64(-76.0), np.float64(-1.5446163880149817)), (np.float64(-76.0), np.float64(-1.5271630954950384)), (np.float64(-75.
0), np.float64(-1.5533430342749532)), (np.float64(-75.0), np.float64(-1.53588974175501)), (np.float64(-75.0), np.float64(-1.4922565104551517)), (np.flo
at64(-75.0), np.float64(-1.4748032179352084)), (np.float64(-74.0), np.float64(-1.562069680534925)), (np.float64(-74.0), np.float64(-1.527163095495038
4)), (np.float64(-74.0), np.float64(-1.509709802975095)), (np.float64(-74.0), np.float64(-1.4835298641951802)), (np.float64(-73.0), np.float64(-1.47480
32179352084)), (np.float64(-73.0), np.float64(-1.457349925415265)), (np.float64(-72.0), np.float64(-1.4660765716752369)), (np.float64(-72.0), np.float6
4(-1.4486232791552935)), (np.float64(-71.0), np.float64(-1.509709802975095)), (np.float64(-70.0), np.float64(-1.509709802975095)), (np.float64(-69.0),
np.float64(-1.53588974175501)), (np.float64(-69.0), np.float64(-1.5009831567151235)), (np.float64(-69.0), np.float64(-1.4922565104551517)), (np.float64
(-69.0), np.float64(-1.4748032179352084)), (np.float64(-69.0), np.float64(-1.4398966328953218)), (np.float64(-68.0), np.float64(-1.4922565104551517)),
(np.float64(-68.0), np.float64(-1.4835298641951802)), (np.float64(-67.0), np.float64(-1.5707963267948966)), (np.float64(-67.0), np.float64(-1.562069680
534925)), (np.float64(-67.0), np.float64(-1.4748032179352084)), (np.float64(-66.0), np.float64(-1.4660765716752369)), (np.float64(-66.0), np.float64(-
1.413716694115407)), (np.float64(-65.0), np.float64(-1.5533430342749532)), (np.float64(-65.0), np.float64(-1.4748032179352084)), (np.float64(-65.0), n
p.float64(-1.457349925415265)), (np.float64(-64.0), np.float64(-1.562069680534925)), (np.float64(-64.0), np.float64(-1.5533430342749532)), (np.float64
(-64.0), np.float64(-1.5446163880149817)), (np.float64(-64.0), np.float64(-1.4660765716752369)), (np.float64(-64.0), np.float64(-1.3962634015954636)),
```

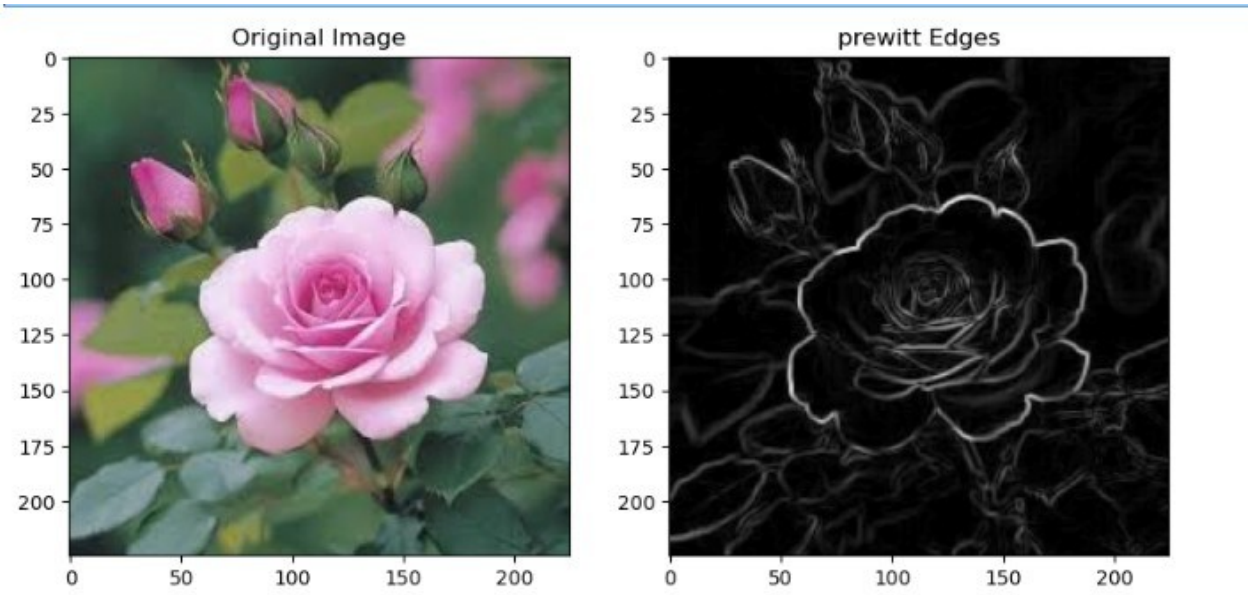
img=cv2.imread("Downloads/rose.jpg") for rho,theta in lines: a=np.cos(theta) b=np.sin(theta) x0=a*rho y0=b*rho x1=int(x0+1000*(-b)) y1=int(y0+1000*(a)) x2=int(x0-1000*(-b)) y2=int(y0-1000*(a)) cv2.line(img,(x1,y1),(x2,y2),(0,0,255),2) plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)) plt.title("Line detection") plt.show() print(lines)



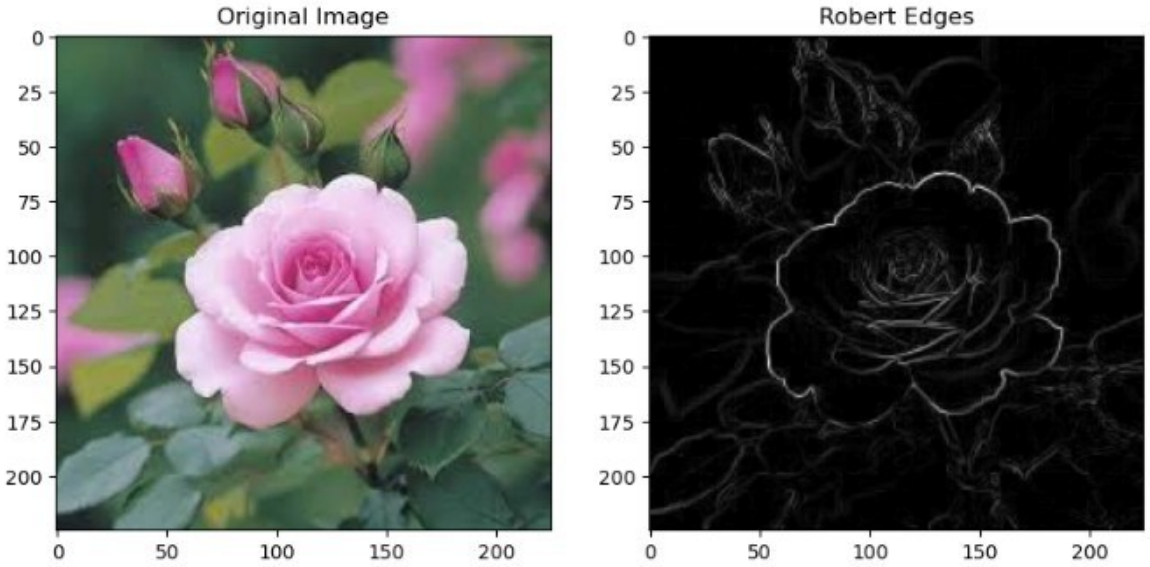

```
[(np.float64(-105.0), np.float64(-1.53588974175501)), (np.float64(-88.0), np.float64(-1.5707963267948966)), (np.float64(-87.0), n
1552935)), (np.float64(-86.0), np.float64(-1.5533430342749532)), (np.float64(-85.0), np.float64(-1.5707963267948966)), (np.float6
1.53588974175501)), (np.float64(-81.0), np.float64(-1.5533430342749532)), (np.float64(-81.0), np.float64(-1.53588974175501)), (np
oat64(-1.5446163880149817)), (np.float64(-80.0), np.float64(-1.5271630954950384)), (np.float64(-78.0), np.float64(-1.5358897417550
0), np.float64(-1.5271630954950384)), (np.float64(-77.0), np.float64(-1.4049900478554354)), (np.float64(-76.0), np.float64(-1.570
loat64(-76.0), np.float64(-1.5446163880149817)), (np.float64(-76.0), np.float64(-1.5271630954950384)), (np.float64(-75.0), np.flo
32)), (np.float64(-75.0), np.float64(-1.53588974175501)), (np.float64(-75.0), np.float64(-1.4922565104551517)), (np.float64(-75.0
```

img=cv2.imread("Downloads/rose.jpg") gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) px=np.array([[-1,0,1],[-1,0,1],[-1,0,1]],dtype=float) py=np.array([[-1,-1,-1],[0,0,0],[1,1,1]],dtype=float)

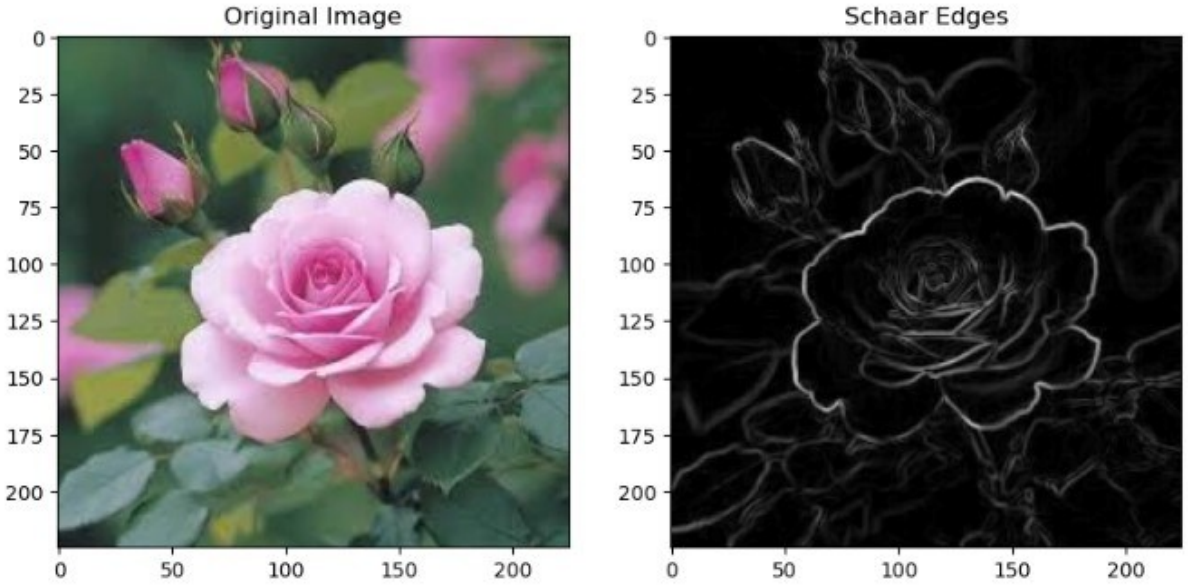
```
Ix=ndimage.convolve(gray.astype(float),px)
Iy=ndimage.convolve(gray.astype(float),py)
edges=np.hypot(Ix,Iy)
edges=np.uint8(255*edges/np.max(edges))
plt.figure(figsize=(10,5)) plt.subplot(121)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))
plt.title("Original Image") plt.subplot(122)
plt.imshow(edges,cmap='gray') plt.title("prewitt Edges")

plt.show()
```




```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) rx=np.array([[1,0],[0,-1]],dtype=float) ry=np.array([[0,1],[-1,0]],dtype=float) Ix=ndimage.convolve(gray.astype(float),rx) Iy=ndimage.convolve(gray.astype(float),ry) edges=np.hypot(Ix,Iy) edges=np.uint8(255*edges/np.max(edges)) plt.figure(figsize=(10,5)) plt.subplot(121) plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)) plt.title("Original Image") plt.subplot(122) plt.imshow(edges,cmap='gray') plt.title("Robert Edges") plt.show()
```

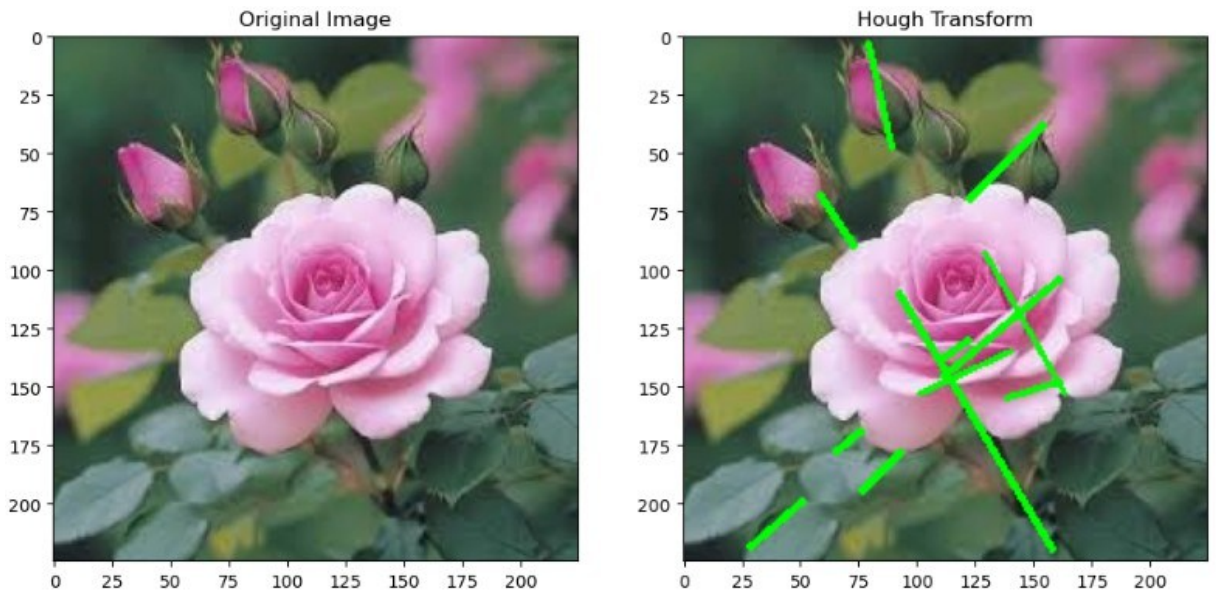



```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
Ix=cv2.Scharr(gray,cv2.CV_64F,1,0)
Iy=cv2.Scharr(gray,cv2.CV_64F,0,1) edges=cv2.magnitude(Ix,Iy)
edges=np.uint8(255*edges/np.max(edges))
plt.figure(figsize=(10,5)) plt.subplot(121)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))
plt.title("Original Image") plt.subplot(122)
plt.imshow(edges,cmap='gray') plt.title("Schaar Edges")
plt.show()
```



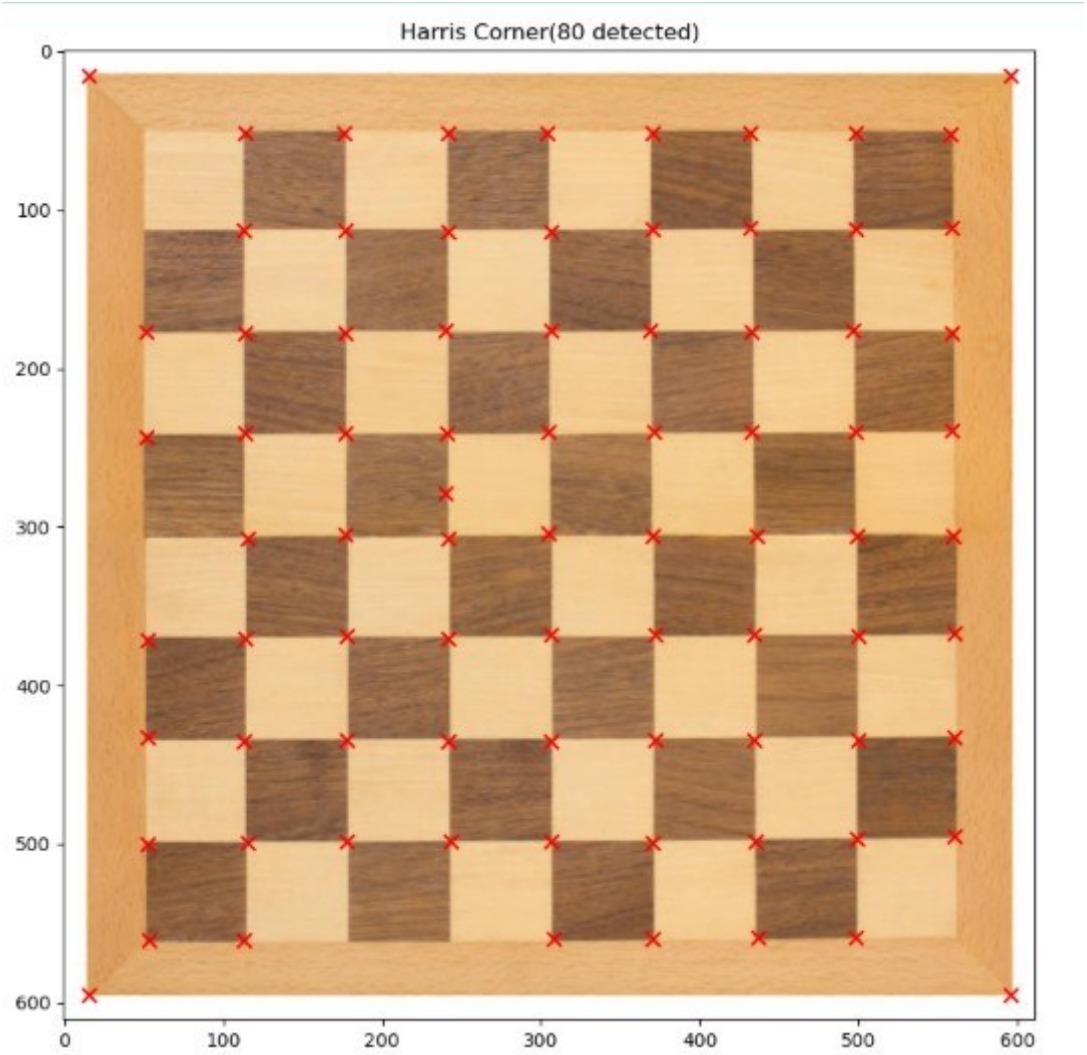

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
gray=cv2.GaussianBlur(gray,(5,5),0)


edges=cv2.Canny(gray,80,150)
```

```
lines=cv2.HoughLinesP(edges,1,np.pi/180,threshold=50,minLineLength=10,maxLineGap=10)

line_img=img.copy()

if lines is not None:
    for x1,y1,x2,y2 in lines[:,0]:
        cv2.line(line_img,(x1,y1),(x2,y2),(0,255,0),2)

plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))
plt.title("Original Image")

plt.subplot(1,2,2) plt.imshow(cv2.cvtColor(line_img,cv2.COLOR_BGR2RGB))
plt.title("Hough Transform")

plt.show()
```




```
img=cv2.imread("Downloads/chess.jpg")
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY).astype(np.float32)/255.0

Ix=ndimage.sobel(gray,axis=1)
Iy=ndimage.sobel(gray,axis=0) sigma=1.5
```

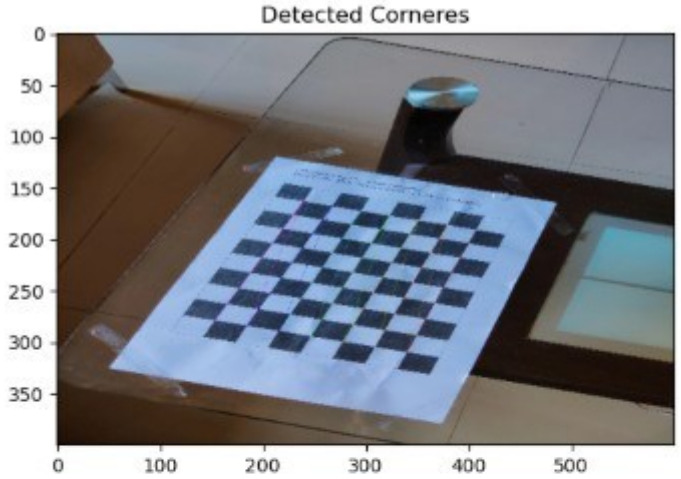
```
Ix2=ndimage.gaussian_filter(Ix**2,sigma)

Iy2=ndimage.gaussian_filter(Iy**2,sigma)

Ixy=ndimage.gaussian_filter(Ix*Iy,sigma)


k=0.05 det=Ix2*Iy2-Ixy**2
trace=Ix2+Iy2

R=det-k*trace**2

R_thresh=np.where(R>0.15*R.max(),R,0)
R_max=ndimage.maximum_filter(R_thresh,size=10)
local_max=(R_thresh==R_max)&(R_thresh>0)
corners=np.argwhere(local_max) plt.figure(figsize=(8,8))

plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))

plt.scatter(corners[:,1],corners[:,0],c='red',s=60,marker='x',linewidths=1.5)
plt.title(f"Harris Corner({len(corners)} detected)") plt.tight_layout() plt.show()
```

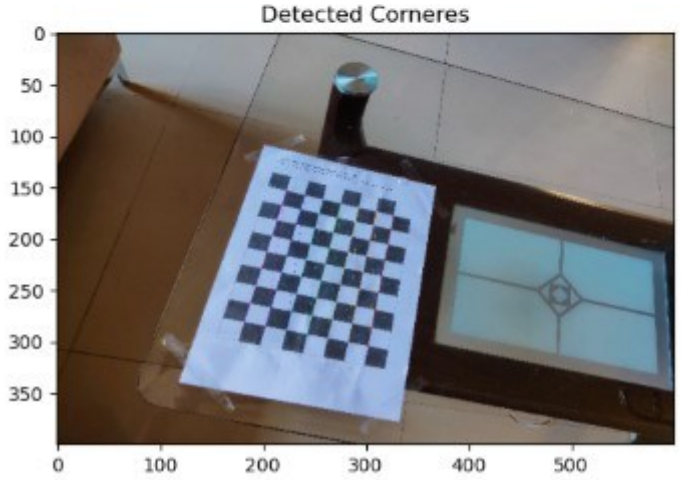

Harris Corner(80 detected)
0
100
200
300
400
500
600
0
100
200
300
400
500
600

EXPERIMENT 4:

```
import cv2 import numpy as np
import glob import os import
matplotlib.pyplot as plt
num_squares_x=8
num_squares_y=10
corners_x=num_squares_x-1
corners_y=num_squares_y-1
checker_size=0.020

path="Downloads/exp 4 calibration"

criteria=(cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER,30,0.001)
objp=np.zeros((corners_x*corners_y,3),np.float32)
objp[:,:2]=np.mgrid[0:corners_y,0:corners_x].T.reshape(-1,2) objp=objp*checker_size
objpoints=[] imgpoints=[]

images=glob.glob(os.path.join(path,'*jpg')) for
fname in images:

   img=cv2.imread(fname)     gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret,corners=cv2.findChessboardCorners(gray,(corners_y,corners_x),None)     if ret:

       objpoints.append(objp)        corners2=cv2.cornerSubPix(gray,corners,(11,11),(-
1,-1),criteria)        imgpoints.append(corners2)
cv2.drawChessboardCorners(img,(corners_y,corners_x),corners2,ret)
img_small=cv2.resize(img,(600,400))        plt.figure(figsize=(6,4))

       plt.imshow(cv2.cvtColor(img_small,cv2.COLOR_BGR2RGB))        plt.title("Detected Corneres")
plt.show() ret,camera_matrix,dist_coeff,rvecs,tvecs=cv2.calibrateCamera(objpoints,imgpoints,gray.shape[::

-1],None,None) print("Intrinsic
Parameters") print("Camera
Matrix:") print(camera_matrix)


print("\nDistortion Coefficients:") print(dist_coeff)
print("\n Extrinsic Parameters") for i,(rvec,tvec) in
enumerate(zip(rvecs,tvecs)):

   R,_=cv2.Rodrigues(rvec)
print("\nImage",i+1)
print("Rotation Vector:")
print(rvec)     print("Rotation
Matrix:")     print(R)
print("Translation Vector:")
print(tvec)
```

cv2.destroyAllWindows()

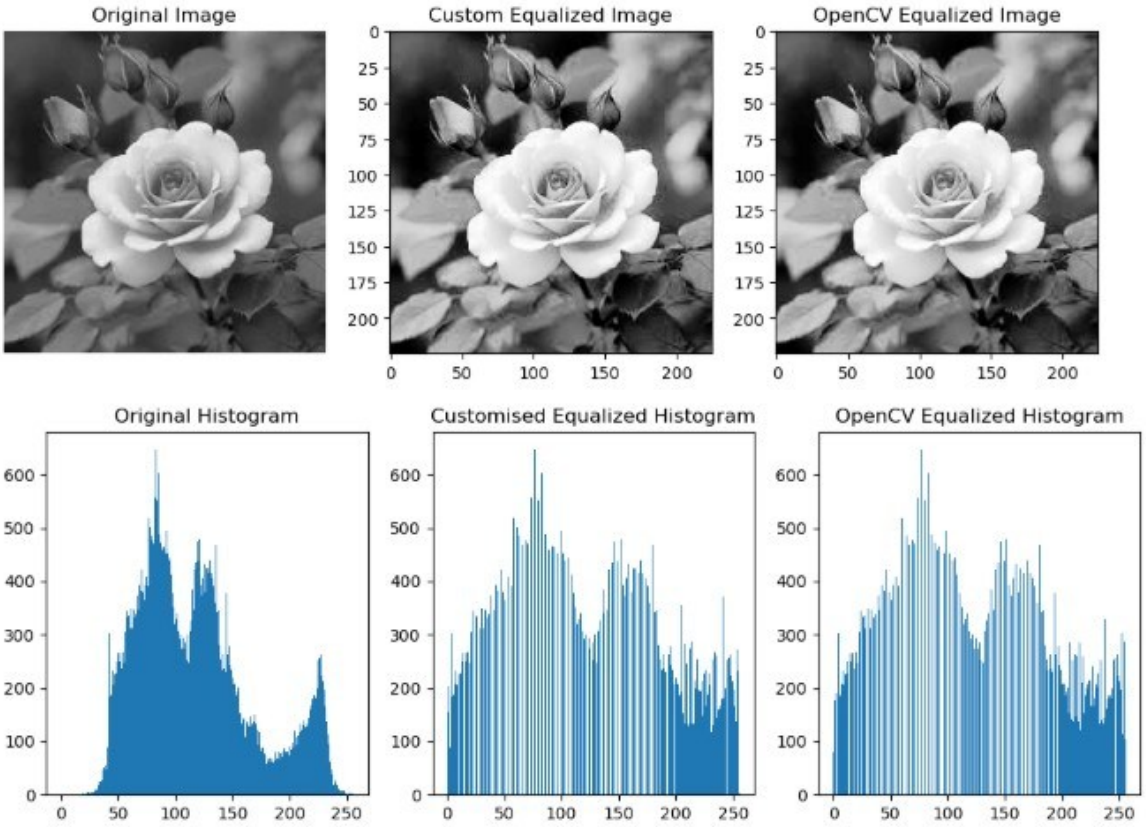




```
Intrinsic Parameters
Camera Matrix:
[[6.06142280e+03 0.00000000e+00 1.53669513e+03]
 [0.00000000e+00 7.03223983e+03 1.67954643e+03]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]

Distortion Coefficients:
[[-2.29367937e+00  3.17493127e+01  7.54764627e-02 -6.42641241e-02
  -1.91333232e+02]]

 Extrinsic Parameters

Image 1
Rotation Vector:
[[-0.49775127]
 [ 0.56058496]
 [ 1.81061024]]
Rotation Matrix:
[[-0.29017477 -0.95515612 -0.05895248]
 [ 0.75474227 -0.26629335  0.59954313]
 [-0.58835594  0.12947836  0.79816831]]
Translation Vector:
[[0.06858559]
 [0.02715072]
 [0.8154804 ]]

Image 2
Rotation Vector:
[[-0.43306214]
 [ 0.55401143]
 [ 1.74435523]]
Rotation Matrix:
[[-0.23582964 -0.97179269  0.00183061]
 [ 0.79476095 -0.19178379  0.57582464]
 [-0.55923109  0.13725141  0.81757118]]
Translation Vector:
[[ 0.10904785]
 [-0.05096435]
 [ 0.96078655]]
```

EXPERIMENT 5:

```
import numpy as np import cv2 import matplotlib.pyplot as plt
image=cv2.imread("Downloads/rose.jpg",cv2.IMREAD_GRAYSCALE) if image is
None:
    print("Image not found")     exit()


values,counts=np.unique(image,return_counts=True)
total_pixels=image.size pdf=counts/total_pixels
cdf=np.cumsum(pdf)
transform=np.floor(255*cdf).astype(np.uint8)
equalized=np.zeros_like(image) for i,v in enumerate(values):
equalized[image==v]=transform[i]
opencv=cv2.equalizeHist(image) plt.figure(figsize=(12,4))
plt.subplot(1,3,1) plt.imshow(image,cmap='gray')
plt.title("Original Image") plt.axis("Off") plt.subplot(1,3,2)
plt.imshow(equalized,cmap='gray') plt.title("Custom
Equalized Image") plt.subplot(1,3,3)
plt.imshow(opencv,cmap='gray') plt.title("OpenCV Equalized
Image") plt.show() plt.figure(figsize=(12,4)) plt.subplot(1,3,1)
plt.hist(image.ravel(),bins=256,range=[0,256])
plt.title("Original Histogram")


plt.subplot(1,3,2)
plt.hist(equalized.ravel(),bins=256,range=[0,256])
plt.title("Customised Equalized Histogram") plt.subplot(1,3,3)
plt.hist(opencv.ravel(),bins=256,range=[0,256])
plt.title("OpenCV Equalized Histogram")

plt.show()
```


Original Image
Custom Equalized Image
OpenCV Equalized Image
Original Histogram
Customised Equalized Histogram
OpenCV Equalized Histogram

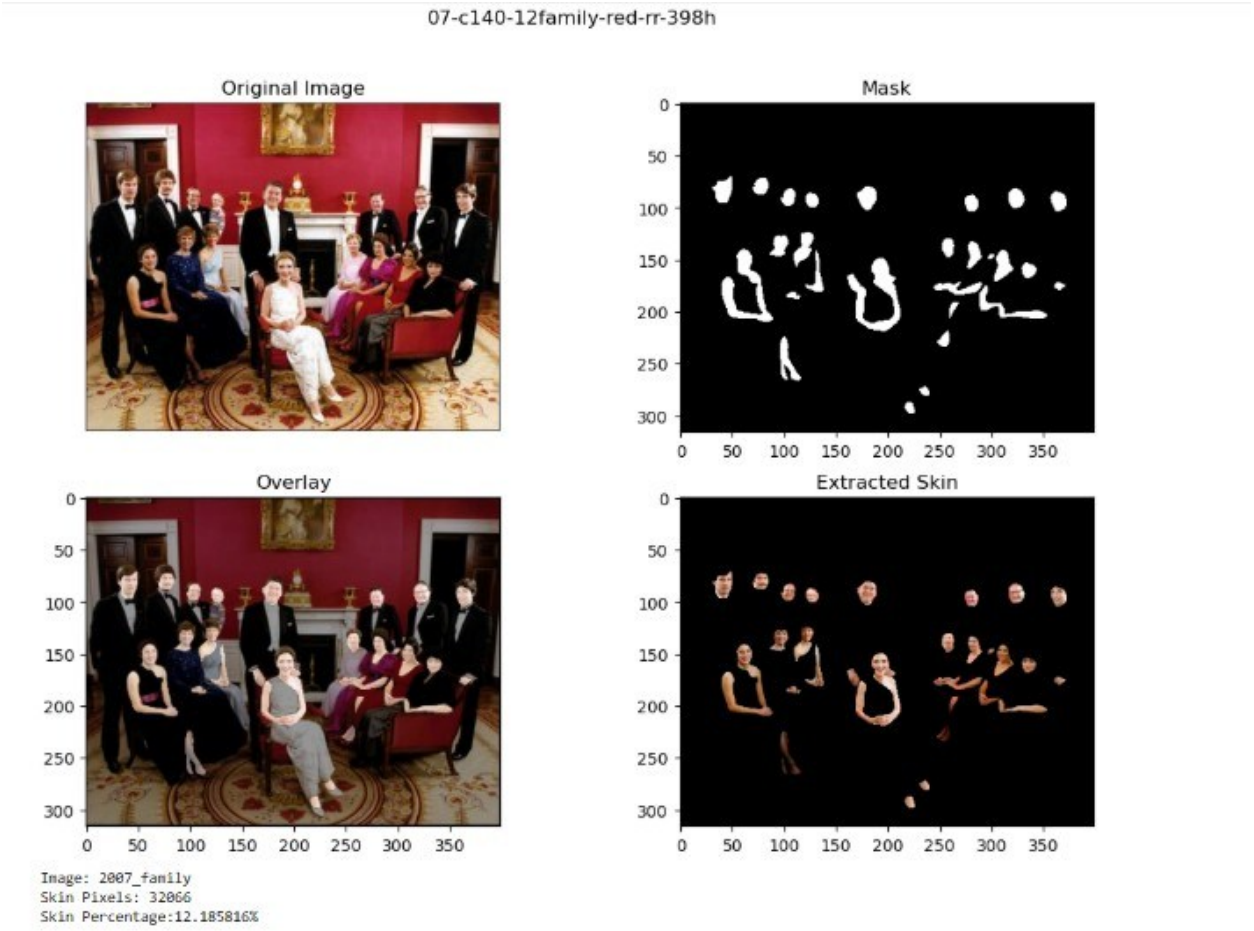
EXPERIMENT 6:

```
import os import numpy as np
import matplotlib.pyplot as plt
import imageio.v2 as imageio

%matplotlib inline


face_dir="Downloads/SkinColor/SkinColor/FamilyPhoto"
mask_dir="Downloads/SkinColor/SkinColor/GroundFamilyPhoto"


images=[] masks=[]
names=[]


image_files=os.listdir(face_dir) mask_files=os.listdir(mask_dir)


for img_file in image_files:

   name=os.path.splitext(img_file)[0]


   for mask_file in mask_files:

      mask_name=os.path.splitext(mask_file)[0]


      if name==mask_name:

         img_path=os.path.join(face_dir,img_file)
mask_path=os.path.join(mask_dir,mask_file)


         image=imageio.imread(img_path)
mask=imageio.imread(mask_path)


         images.append(image)          masks.append(mask)

         names.append(name)   # FIX ADDED


         break


print("Number of Face Images Loaded:",len(images))
```

```
for i in range(len(images)):
    image=images[i]     mask=masks[i]
name=names[i]
mask_rgb=mask[:,:,:3]

    skin_pixels=np.all(mask_rgb==[255,255,255],axis=-1)

    skin_image=np.zeros_like(image)
skin_image[skin_pixels]=image[skin_pixels]

    total_pixels=skin_pixels.size
skin_count=np.sum(skin_pixels)

    skin_percentage=(skin_count/total_pixels)*100

    print("Image:",name)     print("Skin Pixels:",skin_count)
print("Skin Percentage:%2f%%"%skin_percentage)
    print("-------------------------")

    plt.figure(figsize=(12,8))

    plt.subplot(2,2,1)     plt.imshow(image)
plt.title("Original Image")     plt.axis("off")

    plt.subplot(2,2,2)     plt.imshow(mask)
plt.title("Mask")

    plt.subplot(2,2,3)     plt.imshow(image)
plt.imshow(mask,alpha=0.4)
plt.title("Overlay")

    plt.subplot(2,2,4)     plt.imshow(skin_image)
plt.title("Extracted Skin")

    plt.suptitle(name)     plt.show()
```

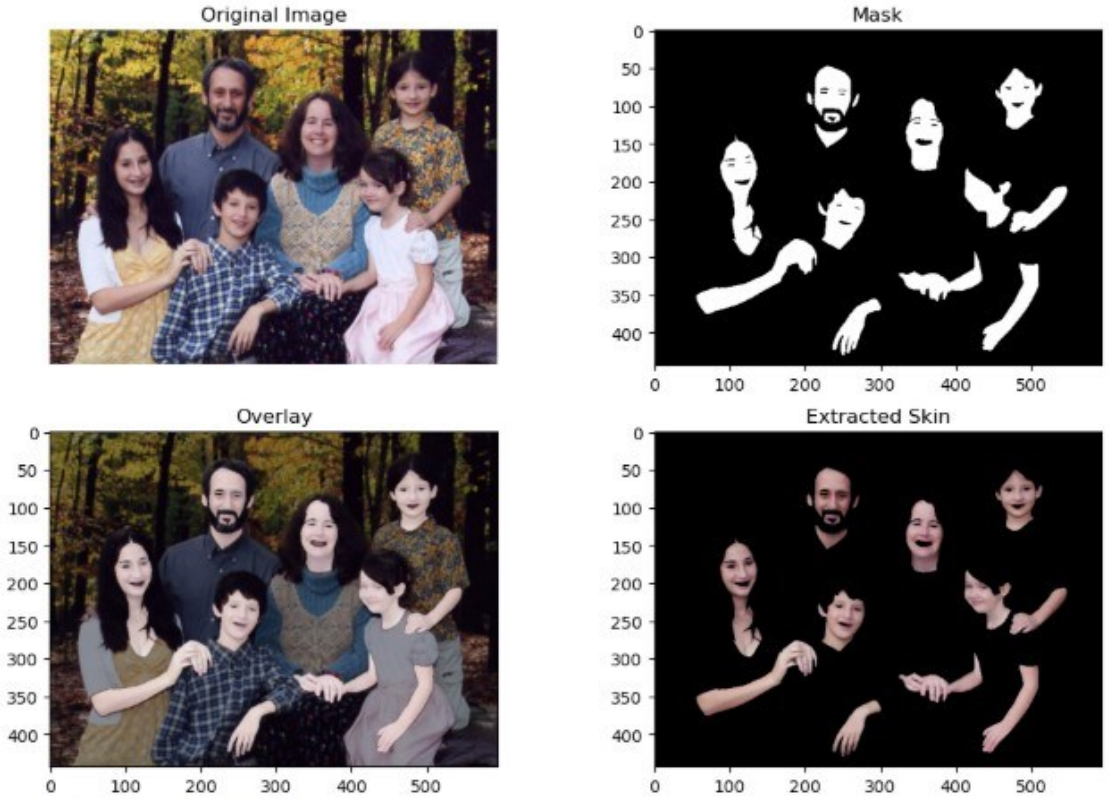
if i==2:
break

```
Number of Face Images Loaded: 46
Image: 07-c140-12family-red-rr-398h
Skin Pixels: 7574
Skin Percentage:6.007106%
------------------------
```




```
Image: 2007_family
Skin Pixels: 32066
Skin Percentage:12.185816%
```

2007_family

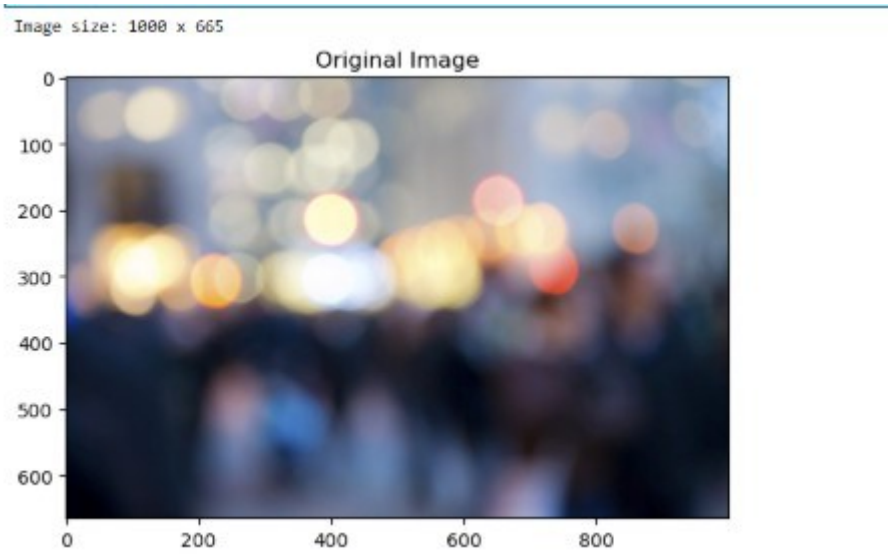


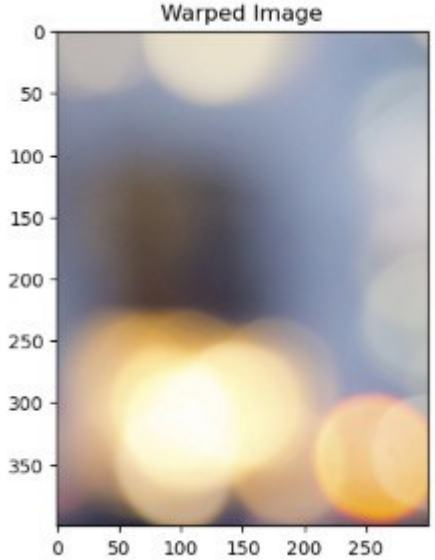

```
Image: 3115267-My-very-large-Indian-family-2
Skin Pixels: 24469
Skin Percentage:9.062593%
------------------------
```

3115267-My-very-large-Indian-family-2

EXPERIMENT 7:

```
import cv2 import
numpy as np

from matplotlib import pyplot as plt


img=cv2.imread("Downloads/warp.jpg")
h,w=img.shape[:2] print("Image size:",w,"x",h)

img_rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb) plt.title("Original Image") plt.title("Original
Image") plt.show()
src_pts=np.float32([[40,60],[250,50],[260,350],[30,360]])
width=300 height=400

dst_pts=np.float32([[0,0],[width,0],[width,height],[0,height]])
H=cv2.getPerspectiveTransform(src_pts,dst_pts)
warped=cv2.warpPerspective(img,H,(width,height))
warped_rgb=cv2.cvtColor(warped,cv2.COLOR_BGR2RGB)
plt.imshow(warped_rgb) plt.title("Warped Image") plt.show()
```




Warped Image
0
50
100
150
200
250
300
350
0
50
100
150
200
250

EXPERIMENT 8

Program: import cv2 import matplotlib.pyplot as plt from IPython.display import display, clear_output

cap = cv2.VideoCapture("images/motiontracking/car.mp4")

ret, frame1 = cap.read() if not ret:

print("Error: Cannot read video") exit()

prev_gray = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY) prev_gray = cv2.GaussianBlur(prev_gray, (11, 11), 0)

print("Car Motion Tracking Started...")

while True:

ret, frame2 = cap.read() if not ret:
break

gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY) gray = cv2.GaussianBlur(gray, (11, 11), 0)

diff = cv2.absdiff(prev_gray, gray)

_, thresh = cv2.threshold(diff, 20, 255, cv2.THRESH_BINARY)

thresh = cv2.dilate(thresh, None, iterations=2) contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for c in contours: if cv2.contourArea(c) > 1000: x, y, w, h = cv2.boundingRect(c) cv2.rectangle(frame2, (x, y), (x + w, y + h), (0, 255, 0), 2)

frame_rgb = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB) clear_output(wait=True) plt.imshow(frame_rgb) plt.title("Car Motion Tracking") plt.axis('off') display(plt.gcf()) plt.close() prev_gray = gray cap.release() print("Tracking Stopped")

```
    clear_output(wait=True)
    plt.imshow(frame_rgb)
    plt.title("Car Motion Tracking")
    plt.axis('off')
    display(plt.gcf())
    plt.close()

    # Update previous frame
    prev_gray = gray

# Release video
cap.release()
print("Tracking Stopped")
```

Car Motion Tracking

EXPERIMENT 9

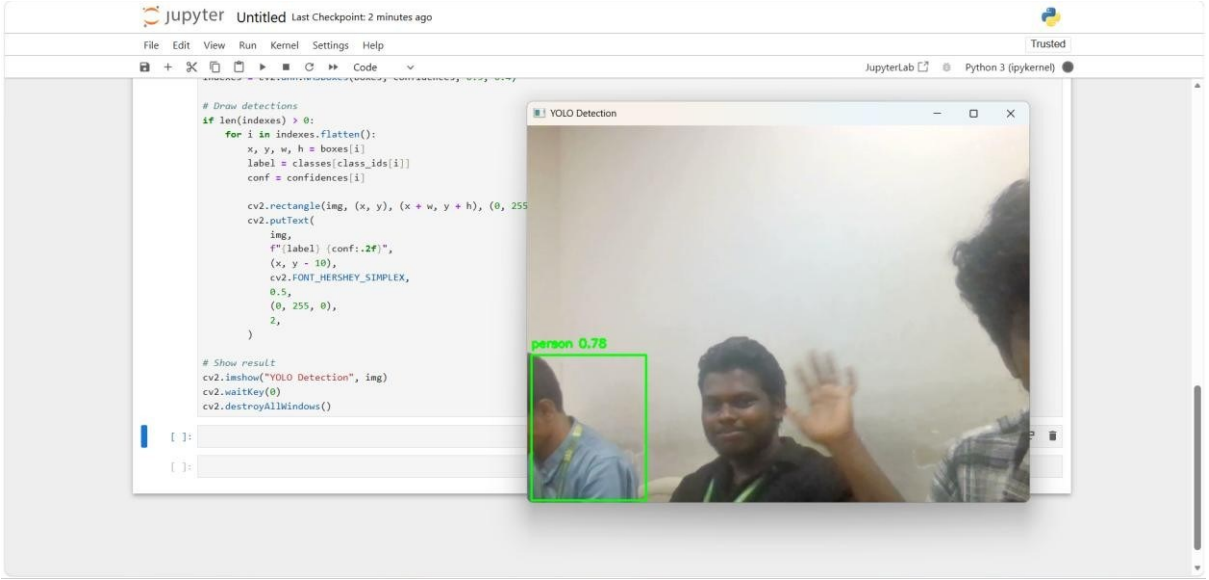
```
import cv2 import numpy as np

net = cv2.dnn.readNet("yolov3-tiny.weights", "yolov3tiny.cfg")

classes = open("coco.names").read().splitlines() img = cv2.imread("images/person.jpg")


if img is None:

print("Image not found") exit()


height, width = img.shape[:2]

blob = cv2.dnn.blobFromImage(img, 1/255, (416,416), swapRB=True) net.setInput(blob)

layer_names = net.getLayerNames() output_layers = [layer_names[i - 1] for i in
net.getUnconnectedOutLayers()] outputs = net.forward(output_layers) boxes = []
confidences = [] class_ids = []


for out in outputs: for det in out: scores = det[5:] class_id =
np.argmax(scores) confidence = scores[class_id]


if confidence > 0.5:

cx, cy, w, h = (det[0:4] * [width, height, width, height]).astype("int")


x = int(cx - w/2) y = int(cy - h/2)


boxes.append([x, y, w, h]) confidences.append(float(confidence)) class_ids.append(class_id)

indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5,
0.4) for i in indexes:


i = i[0] if isinstance(i, (list, tuple, np.ndarray)) else i x, y, w, h = boxes[i] label = classes[class_ids[i]] conf
= confidences[i]
```

cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0), 2) cv2.putText(img, f"{label} {conf:.2f}",

(x, y-5), cv2.FONT_HERSHEY_SIMPLEX,

0.5, (0,255,0), 2)

cv2.imshow("YOLO Detection", img) cv2.waitKey(0) cv2.destroyAllWindows()



EXPERIMENT 10

```
import cv2 import numpy as np import matplotlib.pyplot as plt

imgL = cv2.imread("images/stereo/left.png") imgR = cv2.imread("images/stereo/right.png")

if imgL is None or imgR is None:
print("Error: Images not found in 'images/' folder") exit()

imgL = cv2.cvtColor(imgL, cv2.COLOR_BGR2GRAY) imgR = cv2.cvtColor(imgR, cv2.COLOR_BGR2GRAY)

imgR = cv2.resize(imgR, (imgL.shape[1], imgL.shape[0]))

stereo = cv2.StereoSGBM_create( minDisparity=0, numDisparities=16 * 5,
blockSize=5,
P1=8 * 3 * 25, P2=32 * 3 * 25, uniquenessRatio=10,
speckleWindowSize=100, speckleRange=32
)

disparity = stereo.compute(imgL, imgR)

disp = cv2.normalize(disparity, None, 0, 255,
cv2.NORM_MINMAX)
disp = np.uint8(disp)

depth = np.zeros_like(disparity, dtype=np.float32) depth[disparity > 0] = 1000 / (disparity[disparity > 0] +
1)
```

plt.figure(figsize=(10,4)) plt.subplot(1,3,1) plt.title("Left Image") plt.imshow(imgL, cmap='gray')

plt.subplot(1,3,2) plt.title("Right Image") plt.imshow(imgR, cmap='gray')

plt.subplot(1,3,3) plt.title("Depth Map") plt.imshow(disp, cmap='jet')

plt.show()

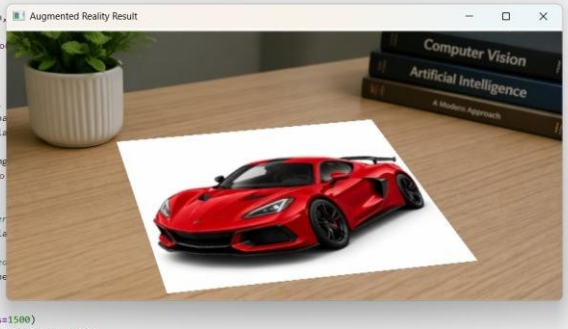
valid_depth = depth[depth > 0] print("\n===== DEPTH SUMMARY =====") print(f"Avg Depth : {np.mean(valid_depth):.2f}") print(f"Min Depth : {np.min(valid_depth):.2f}") print(f"Max Depth : {np.max(valid_depth):.2f}") print("==========\n")

```
if valid_depth.size > 0:
    print(f"Avg Depth : {np.mean(valid_depth):.2f}")
    print(f"Min Depth : {np.min(valid_depth):.2f}")
    print(f"Max Depth : {np.max(valid_depth):.2f}")
else:
    print("No valid depth values found")
print("==========\n")
```

```
===== DEPTH SUMMARY =====
Avg Depth : 4.02
Min Depth : 0.79
Max Depth : 100.00
==========
```

[ ]:

EXPERIMENT 11

```
import cv2 import numpy as np
ref_img = cv2.imread("images/AR/reference.jpg", 0) # query image (grayscale)
scene_img = cv2.imread("images/AR/scene.jpg") overlay_img = cv2.imread("images/AR/overlay.png") if
ref_img is None or scene_img is None or overlay_img is None:
print("Error: Check image paths") exit()
overlay_img = cv2.resize(overlay_img, (ref_img.shape[1], ref_img.shape[0]))


orb = cv2.ORB_create(nfeatures=1000) kp1, des1 = orb.detectAndCompute(ref_img, None) kp2, des2 =
orb.detectAndCompute(cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY),
None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True) matches = bf.match(des1,
des2)
matches = sorted(matches, key=lambda x: x.distance) good_matches = matches[:50] if len(good_matches)
< 10:
print("Not enough matches found") exit()
src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1,1,2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1,1,2)
H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0) h, w = ref_img.shape
overlay_warp = cv2.warpPerspective(overlay_img, H,
(scene_img.shape[1], scene_img.shape[0])) mask_overlay = np.zeros(scene_img.shape,
dtype=np.uint8)

mask_overlay[overlay_warp > 0] = 255 result = cv2.addWeighted(scene_img, 1,
overlay_warp, 1, 0)
cv2.imshow("Augmented Reality Result", result) cv2.waitKey(0) cv2.destroyAllWindows()
```

```python
# Paths
ref_path = "images/AR/reference.jpg"
scene_path = "images/AR/scene.jpg"
overlay_path = "images/AR/overlay.png"

# Check files
for p in [ref_path, scene_path,
    if not os.path.exists(p):
        print(f"❌ File not fo
        exit()

# Load images
ref_img = cv2.imread(ref_path,
scene_img = cv2.imread(scene_pa
overlay_img = cv2.imread(overla

if ref_img is None or scene_img
    print("❌ Error: Failed to
    exit()

# Resize overlay to match refer
overlay_img = cv2.resize(overla

# Convert scene to gray for fea
scene_gray = cv2.cvtColor(scene

# ORB feature detection
orb = cv2.ORB_create(nfeatures=1500)
kp1, des1 = orb.detectAndCompute(ref_img, None)
kp2, des2 = orb.detectAndCompute(scene_gray, None)

if des1 is None or des2 is None:
    print("❌ Could not find descriptors")
    exit()
```